Agenthon 2026 · NeurIPS 2026 Competition Track · Track 3: Accelerated Market Simulation

3rd of 55 teams

The Clock Starts Before Your Code Does

fastsim: a row-exact Rust rewrite of ABIDES, tuned from the event loop to the container start

Peidong Liu · Yiping Yin · Xing Zheng · Team abc123

github.com/EscapedShark/agenthon2026-t3-neurips

3.25M
rows/s Final score, 0.004% behind 2nd place

×880
median speedup over ABIDES, same machine

0
row mismatches in 2,969 scenarios at release

98%
of scored time is outside our program

1.8 MB
whole image: one static binary, empty root

1 Task and score

Task. Replace ABIDES, a Python discrete-event simulator of a limit-order-book market, with something faster. For any scenario, the trace and the message ledger must match the baseline in every row, column and dtype.

Model. One exchange; noise, value, momentum and market-making agents; a mean-reverting fundamental with jumps; stochastic message latency. The public set has 71 units, from 604 to 1.17M rows.

rate = verified rows ÷ (FinishedAt − StartedAt)
unit score = median of 4 timed runs
Final score = mean of unit scores

- **Exactness.** One wrong row fails the unit.
- **Repeatability.** Every repeat must be byte-identical, or the entry is refused.
- **Machine.** Xeon Platinum 8570 VM, 4-CPU quota, 16 GiB, NVIDIA B200 attached, runc.

2 Where the clock goes

One floor-image repeat, organizers' split, ms

146	64	165	166	73

only this is scored

Median public unit, 72,036 rows
floor 170 ms, 98% of the window — **4.2 ms**

Largest public unit, 1,168,360 rows
floor 170 ms, 78% of the window — **47 ms**

Docker stamps StartedAt just before the task is created, so runc's GPU hooks run on the clock. A static binary that only copies outputs takes 170 ms (142–204) on the Final machine.

Rates are averaged, so big units dominate: 5 of 71 public units hold 39% of a public-set score. At this floor, 10% less program time adds 0.85%, and 1 ms less on every unit adds 0.54%.

3.25M rows/s on this floor needs ≥ 0.55M rows per Final unit, 5× the public mean. Program: release 1.26, i7-10700K lab, measured apart from the floor.

3 Match every row

Speed counts only after outputs match.

RNG	NumPy legacy RandomState, bit for bit. `pareto` is exp(E/a) − 1, not expm1
Seeds	The full seed chain, with its hidden exponential draw between two seeds
Order	Time, sender, receiver, message id. Wakeups consume ids too
Numbers	Python's int and float mix, half-even rounding, glibc 2.41 libm
Files	pandas metadata, so nullable `t_send_ns` stays an integer

Verification at release 1.26

95/95	public scenarios identical
2,274	fuzzed ABIDES runs, 0 mismatches
600	boundary runs, 0 mismatches
142/142	container runs pass gates g0–g3
374	Parquet files equal to release 1.25
45	Rust tests, mutation-checked queue

4 System overview: one thread owns the order, everything else moves off it

Input
`scenario.json`
`card.toml`

Run plan
threads pre-started, row groups, memory pre-faulted

Look-ahead helpers latency samples and trader draws, same streams, same order

Event loop, one thread
wakeups + messages + busy groups → min(u128 key) → deliver → match or act → send

Trace replay
raw events to final rows, fill labels

Encode workers
row groups to Parquet column chunks

Output
`trace.parquet`
`message_trace.parquet`
`events.json`

written while the simulation runs, hashed with SHA-256 as written

5 The bug 95/95 could not show



neighbouring doubles, 256 ns apart at t ≈ 1.6×10^{18} ns

A value trader observes 90 ns after a megashock. ABIDES holds these times as `np.float64`, and NumPy rounds the int to a double before comparing, so the shock is not due yet: old value, no trade. fastsim compared exactly, applied the jump and sold at the bid.

Expected hits over the 95 public scenarios: 0.0002. Differential fuzzing found it in 1 of 600 runs, over two weeks after the first 95/95.

6 Inside the event loop

- **Sort key.** Time, sender, receiver and id fit one `u128`, so compares have no branches.
- **Wakeups.** One FIFO per wakeup interval replaces a heap.
- **Messages.** Sorted array, 4-ary heap, or, with 16 or more pending, a 512-bucket ring that retunes its width.
- **Busy receivers.** Waiting messages requeue as one group.
- **Order book.** Search starts at the best price; acks go out during matching, in ABIDES's order.
- **Output.** Our own Parquet writer. `msg_type` goes last: later fills relabel earlier rows.

7 Empty the image

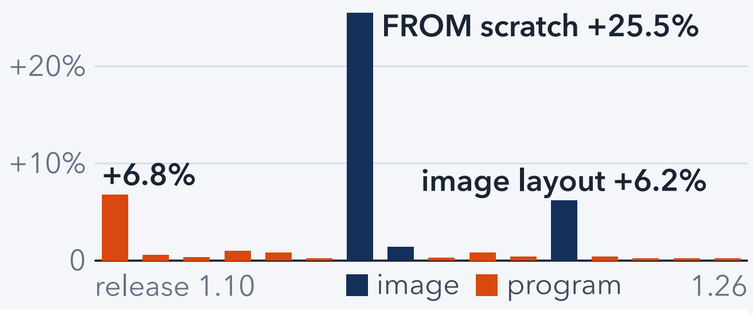
Lab, smallest unit
Debian 13 slim — 162 ms
FROM scratch — 124 ms

Lab, CDI injection
copy-up in window — 110 ms
no copy-up in window — 102 ms

Final machine, organizers
python:3.13-slim — 343 ms
static, empty image — 170 ms

- **Empty root.** The GPU hook runs `ldconfig` in the root, so less root is less work.
- **No copy-up on the clock.** No `/usr` in the image; a config-only `WorkingDir` copies `/etc` at create time.
- **Lean binary.** Static, fixed address, x86-64-v3, PGO with ext-TSP.

8 What each change was worth



Helper look-ahead	×1.59 on log-normal units
Prefault memory	faults 440–765 → ~5
Polling helpers	futex wakes 592 → ~30
x86-64-v3, new PGO	program time −7.2%
No Rust start-up	start-up syscalls 42 → 24
Ring retuning	−7.9% on denser variants

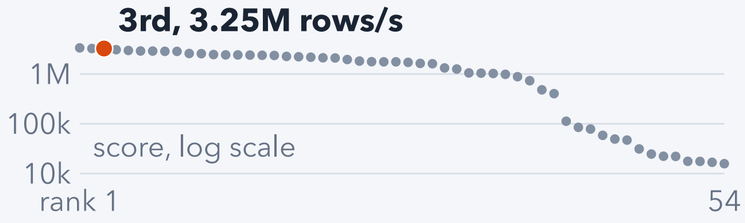
9 Results

Against ABIDES, one machine



Final binary, i7-10700K Linux, 65 scenarios, ×66 to ×1,765. ABIDES ran at 0.96× its official reference time.

Final leaderboard, 54 scored teams



10 Lessons

1. **Read the clock first.** Lab: 3 image changes gave +35%, 13 releases +12%.
2. **Fuzz after you pass.** The last bug hid behind 95/95 public scenarios.
3. **Never tune on a noisy board.** One binary scored 209k to 221k on Dev.
4. **Treat builds as draws.** PGO varies 1–2%; we shipped the best of 12.

Final image, Apache-2.0:
`docker.io/greedyshark/fastsim-t3:1.26`

Timing data: track3-simulation-public #24, #34. Built with Claude Code and Codex. Thanks to the organizers.